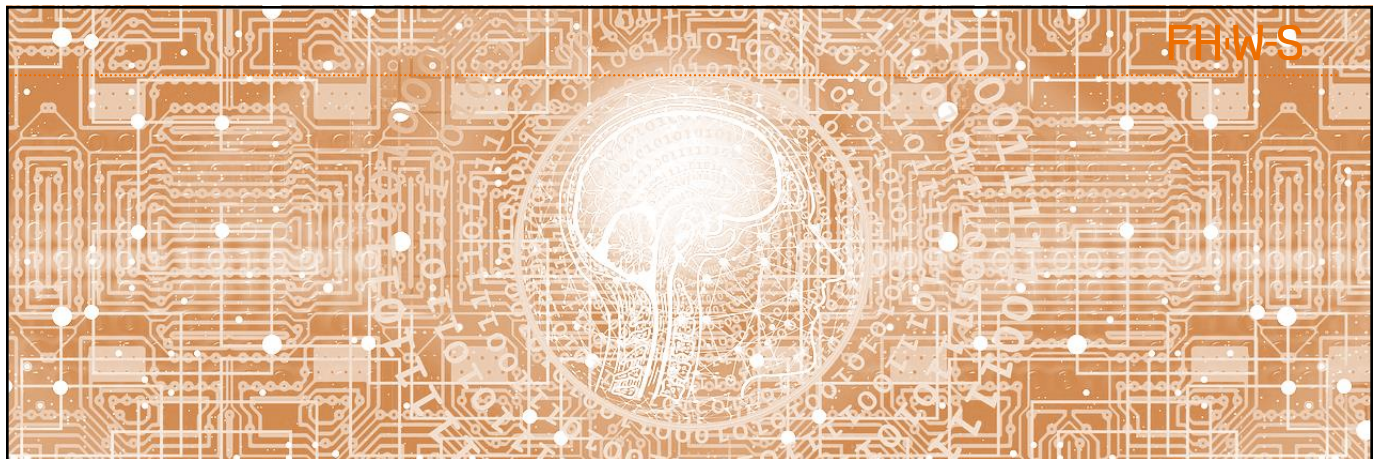




Software Engineering and Cyber Security

Prof. Dr. Marian Daun

FHWS University of Applied Sciences
Würzburg-Schweinfurt

The slide has a background image of a desk with a notebook, a pen, and a smartphone. A semi-transparent white box with an orange border contains the agenda items. The FHWS logo is in the bottom left, and the number '2' is in the bottom right.

Agenda

- Software Engineering Fundamentals
- Requirements Engineering
- Architecture Design
- Modelling
- Development and Implementation
- Software Quality Assurance

FHWS University of Applied Sciences
Würzburg-Schweinfurt

2

Modelling

Part B: Structures

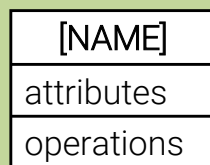
UML Class Diagrams

Class

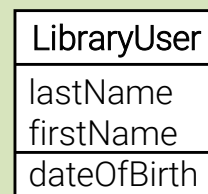
Definition

- A class describes a static structure element of the system.
- A class is represented by a box with compartments.
 - Top compartment: Name of the class (mandatory).
 - Middle compartment: Attributes of the class.
 - Bottom compartment: Methods of the class.
 - ...

Notation



Example

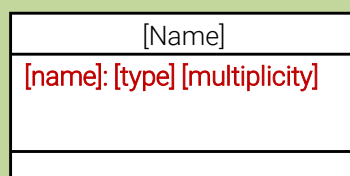


Attribute – Basic Elements

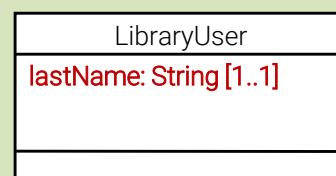
Definition

- “Attribute” represents a **property** of the class.
- Name: Labels the attribute (attribute semantics).
- Type: Defines the data or object type of the attribute values.
- Multiplicity: Assigns the number values for the attribute.

Notation



Example



Attribute – Advanced Elements

Definition

- **Visibility:** Defines literals to determine the visibility of elements in a model.
 - Public (+)
 - private (-)
 - protected (#)
 - derived (/)
- **Default:** Defines a default value.
- **Property-string:** Indicates further characteristics.

Notation

[Name]
[visibility] [name]: [type] [multiplicity] = [default] [{property string}]

Example

LibraryUser
+ lastName: String [1..1] = "New user" {ordered}



Group Discussion

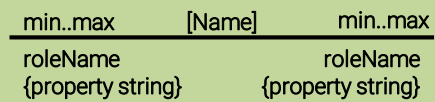
When to use Attributes? And when better not?

Association

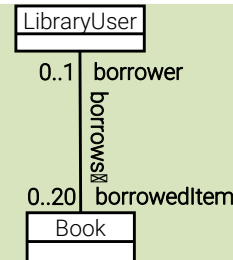
Definition

- "Association" represents a relationship between classes.
- **Name:** Assigns a label to the association (association semantics).
- **Multiplicities:** Assigns a min and max number of participating instances of classes with respect to the related association.
- **RoleName:** Assigns a label indicating a role for the annotated class with respect to the related association.
- **Property String:** Indicates further characteristics.

Notation



Example

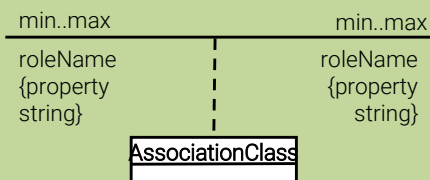


Association Class

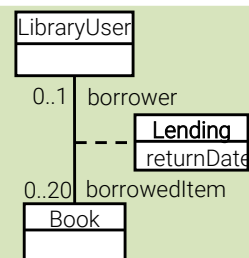
Definition

- Association classes allow to document information related to the existence of the association (i.e. the existence of the corresponding links on the instance level).
- Necessary, if information depends on the association and cannot be related to one of the participating classes.

Notation



Example

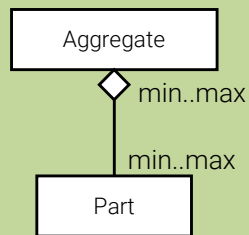


Aggregation

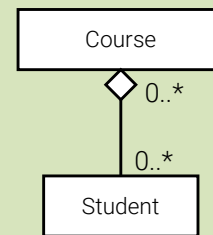
Definition

- A hierarchical relationship between a whole and its parts.
- The lifetime of the parts do not depend on the aggregation relation.
- The aggregation relation may change to different „owners“ (i.e. instances of the aggregating class) over time.
- Cardinalities are assigned to each end of the aggregation.

Notation



Example

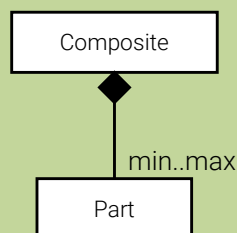


Composition

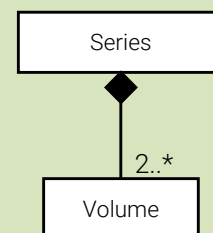
Definition

- Strong „aggregation“ that correlates the lifetime of the parts to the lifetime of the whole, i.e. the parts can not exist without the composite.
- A part can only be involved in one composition relationship at a given time.
- Cardinality on the end of the composite is always one and only one.

Notation



Example

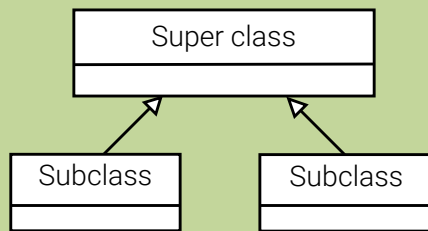


Generalization

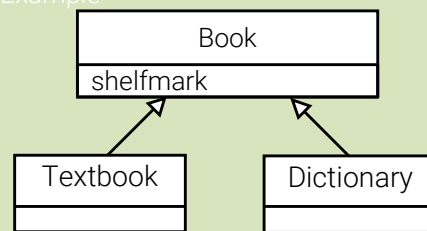
Definition

- A generalization link relates the subclass to the super class.
- A subclass inherits attributes, operations and relationships from the super class.
- A subclass might define additional attributes, operations and relationships and override inherited ones (if required).

Notation

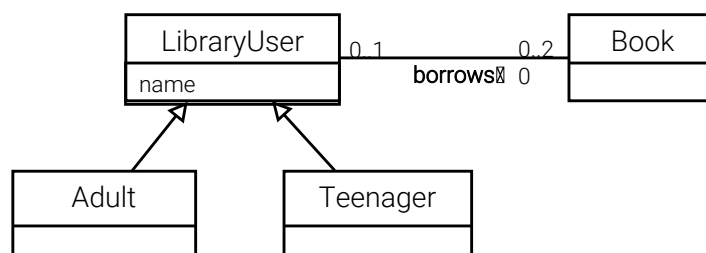


Example



Inheritance and (Re-)Definition of Attributes / Relationships

Attributes and relationships are inherited from the super class.



Subclasses may

- **redefine** attributes and relationships of the super-class.
- define **additional** attributes and relationships.

Generalization Sets

Generalization Set Classification

Disjoint vs. overlapping:

Disjoint: Objects of the super class can belong to only one subclass.

Overlapping: Objects of the super class may belong to two or multiple subclasses.

Complete vs. incomplete:

Complete: Each object of the super class has to belong to at least one subclass.

Incomplete: There may be objects of the super class, which do not belong to any of the subclasses.

Four possible combinations:

Disjoint and complete generalization.

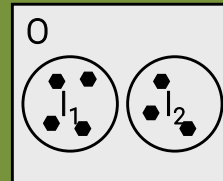
Disjoint and incomplete generalization.

Overlapping and complete generalization.

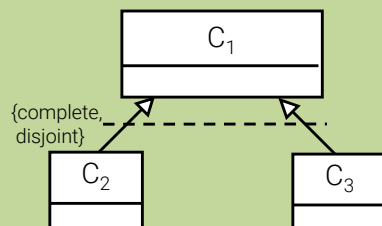
Overlapping and incomplete generalization.

Disjoint and Complete

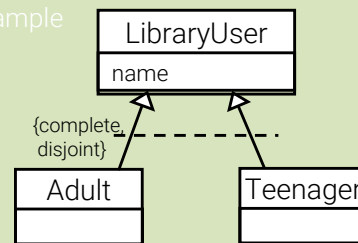
Definition Disjoint and complete indicates that the generalization set is covering and its specific classifiers have no common instances.



Notation



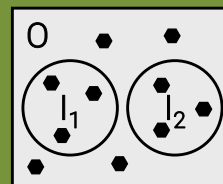
Example



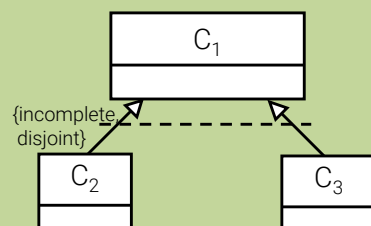
[UML 2011]

Disjoint and Incomplete

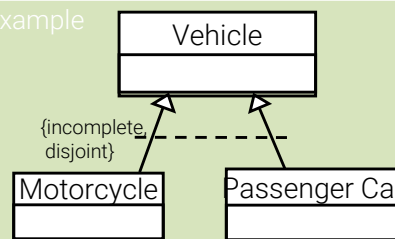
Definition Disjoint and incomplete indicates that the generalization set is not covering and its specific classifiers have no common instances.



Notation



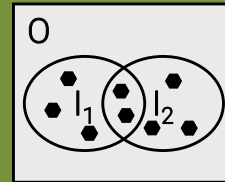
Example



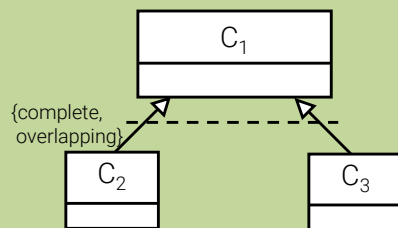
[UML 2011]

Overlapping and Complete

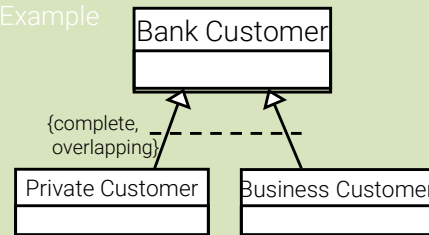
Definition Overlapping and complete indicates that the generalization set is covering and its specific classifiers do share common instances.



Notation



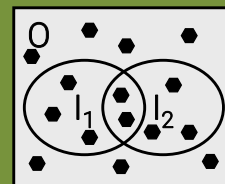
Example



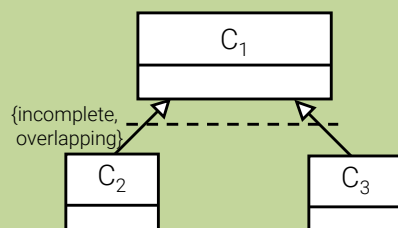
[UML 2011]

Overlapping and Incomplete

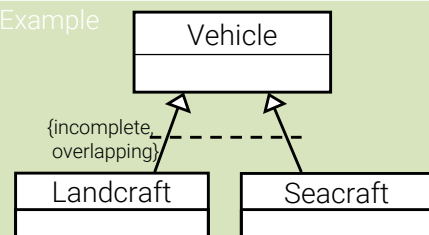
Definition Overlapping and incomplete indicates that the generalization set is not covering and its specific classifiers do share common instances.



Notation



Example

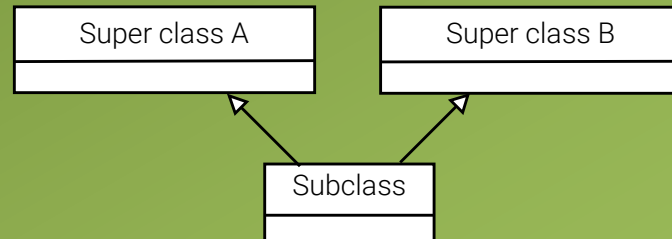


[UML 2011]



Group Discussion

What is this?



Modeling Choices

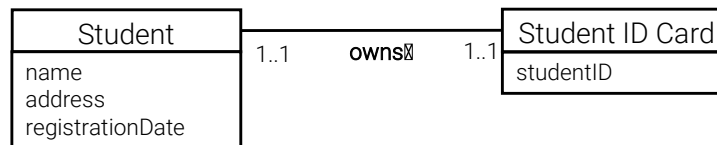
Class vs. Attribute

Classes reflect objects of a specific type relevant for the system.

An [object](#) has an [identity](#) of its own.

[Attributes](#) are only meaningful as [property of a class](#).

A class with only [one attribute](#) and which is related to another class, may be modelled as an [attribute of the other class](#).



Class vs. Attribute

Classes reflect objects of a specific type relevant for the system.

An [object](#) has an [identity](#) of its own.

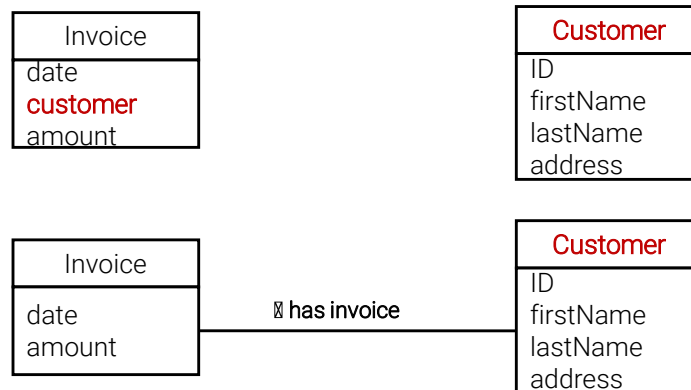
[Attributes](#) are only meaningful as [property of a class](#).

A class with only [one attribute](#) and which is related to another class, may be modelled as an [attribute of the other class](#).



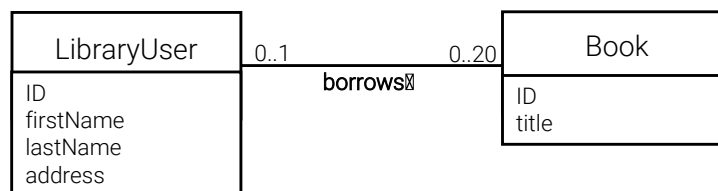
Attribute vs. Relationship Type

If an [attribute](#) of a [class \$C_1\$](#) is a [reference](#) to another [class \$C_2\$](#) , consider modelling the attribute as a [association between \$C_1\$ and \$C_2\$](#) , especially if a similar attribute referencing to C_1 is also defined for C_2 .



Entity type vs. Relationship Type

While objects (of a class) have an [identity of its own](#), a [association](#) draws its [identity](#) from the [participating objects](#)



The identity of the Relationship type "BORROWS" is defined by the identities of the participating LIBRARY USER and BOOK entities.

**Exercise**

Create a UML Class Diagram for a Liquor-Webshop

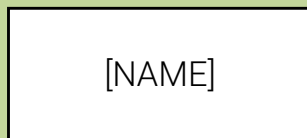
Entity-Relationship Models

Entity Type

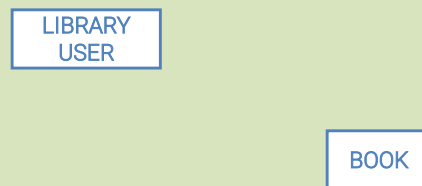
Definition

- Entity type represents a set of entities with similar properties.
- Abstracts from concrete entities (i.e., the instances).
- Entities are material or immaterial objects from the universe of discourse (system context) relevant for a system.

Notation



Example



Relationship Type

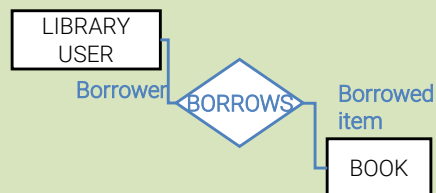
Definition

- Relationship type relates two or multiple entity types to each other
- Represents a set of similar relationship instances that relate entities of the respective entity types and that are relevant for the system

Notation



Example

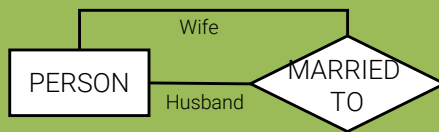


Different Kinds of Relationship Types

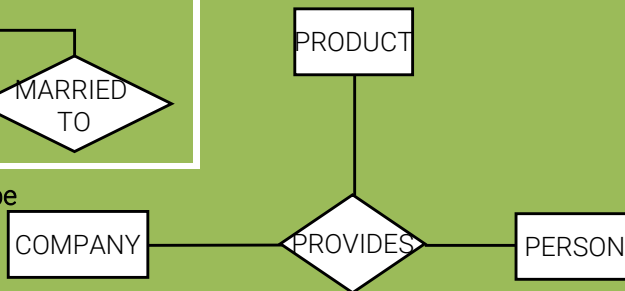
Binary Relationship Type



Reflexive Relationship Type



Ternary Relationship type

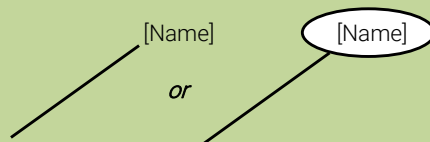


Attribute

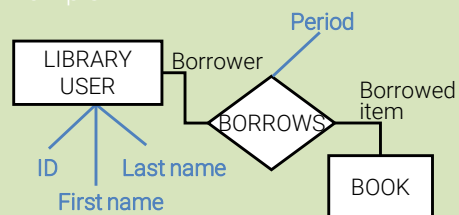
Definition

- An Attribute defines a **property** that is relevant for the system.
- Attributes can be defined for both, entity and relationship types.
- Attributes of relationship types = properties of relationships between entities.

Notation



Example



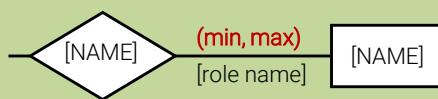
Cardinality Constraint

Definition

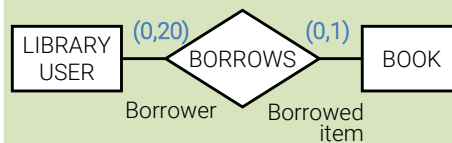
- **Restricts** the permissible **number of entities** (of a certain entity type) that may participate in a defined relationship (of a certain type).
- **(min, max)**: An entity has to participate in at **least min** and **at most max** relationship instances of a certain relationship type.
- **min = 0** denotes an **optional** relationship.

Notation

Notation

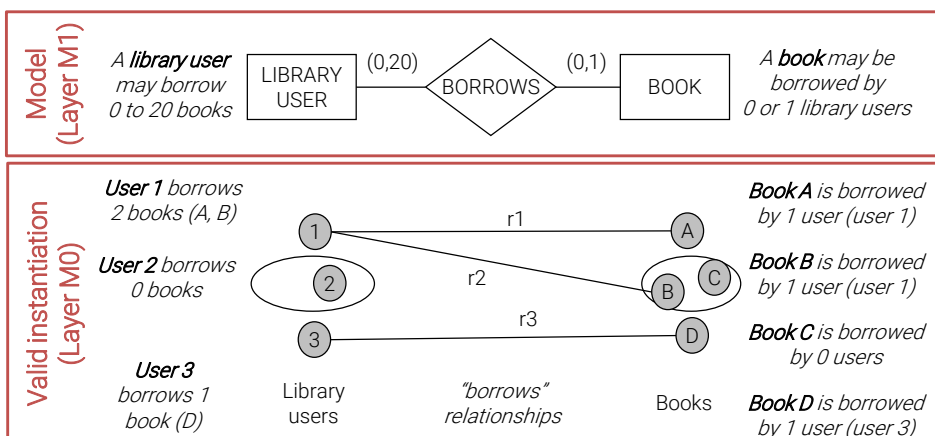


Example



Effects of Cardinality Constraints

Cardinality constraints **restrict** the set of **valid instantiations** of entity relationship models.





Exercise

Create an ER Diagram describing an Exam at THWS and its relations



Group Discussion

How do we derive a database scheme from an ER-diagram?

Literature

- | | |
|--------------------------|--|
| [Chen 1976] | P. Chen: The Entity Relationship Model - Toward a Unified View of Data. In: ACM Transactions on Database Systems, Vol. 1, Issue 1, 1976, pp. 9-36. |
| [Elmasri & Navathe 2010] | R. Elmasri, S.B. Navathe: Fundamentals of Database Systems. 6th ed., Addison-Wesley, 2010. |
| [UML 2011] | Object Management Group: OMG Unified Modelling Language Superstructure. Version 2.4.1, 2011. |
| [Vossen 2008] | G. Vossen: Datenmodelle, Datenbanksprachen und Datenbankmanagementsysteme. 5th ed., Oldenbourg, 2008. |

Questions for Self-Assessment

- What is the difference between an aggregation and a composition in UML class diagrams?
- How can generalization sets be classified and which combinations are possible?
- What are the benefits and problems when defining generalization relations with multiple super-classes?
- What are the modelling constructs of the ER model?
- Explain the cardinality constraints using an own example?

Thanks

UNIVERSITÄT
DUISBURG
ESSEN

Offen im Denken

PALUNO
The Ruhr Institute for Software Technology
<https://www.paluno.uni-due.de>

This Lecture Material is based on Lecture Material provided by
Prof. Dr. Klaus Pohl from **University of Duisburg-Essen**

The original Lecture Material was created by:

Prof. Dr. Klaus Pohl

Torsten Bandyszak

Jennifer Brings

Johanna Buchner

Prof. Dr. Marian Daun

Felix Föcker

Andrea Salmon

Philipp Schmidt

Prof. Dr. Bastian Tenbergen

Prof. Dr. Thorsten Weyer